

Plano de melhoria — Suíte E2E TOTVS Fluig Cassi

Planejamento de implementação dos pontos levantados em docs/revisao-qa-2026-09-03.md . Cada etapa traz **o que mudar, onde, como verificar e qual norma sustenta a escolha**. É a base para a implementação — nada aqui foi executado ainda.

Base	commit 37beb78 · execução completa de 03/09/2026 (233 testes / 162 verdes / 71 vermelhos)
Normas	playwright-test-creator (fluxo 1-18, proibições, quality gate) · fluig-master (fatos da plataforma) · cassi-fluig-master (regras do cliente e do ambiente)
Regra de execução	tudo em primeiro plano, fatiado; medição só com o Protheus sustentando ~845 contratos em 5 amostras (docs/estabilidade-do-ambiente.md)

0. Correções ao overview — leia antes de implementar

Ao preparar este plano, li as **mensagens de erro reais** dos 9 vermelhos do gate (relatorios-2026-09-03/*.json), não só os títulos. Três classificações do overview estavam erradas, e a correção muda o que se implementa:

Teste	Overview dizia	Mensagem real de 03/09	Consequência
criacao-solicitacao.spec.js:513 (CT-ACC-06-S2)	defeito sem @bug	PRÉ-CONDIÇÃO AUSENTE: nenhum contrato com CNB_QUANT vazio e CNB_QTDORI preenchido em 15 tentativas	não receber @bug — README §"A tag @bug " exclui pré-condição explicitamente
payload-solicitacao.spec.js:206 (D-02)	defeito catalogado sem @bug	PRÉ-CONDIÇÃO AUSENTE: o start nunca foi disparado – contrato sem itens	idem
validacoes-faturamento.spec.js:79 (CT-FAT-02-S2)	defeito sem @bug	PRÉ-CONDIÇÃO AUSENTE: nenhuma competência recusada nos contratos amostrados	idem — é defeito quando a massa existe , pré-condição quando não
modal-solicitacao-compra.spec.js:50	sem triagem	TimeoutError: waiting for status /Mostrando/ — a grade não carregou	ambiente; retries: 2 do CI cobre

Placar corrigido dos 9: 7 pré-condição ausente (todos com entrada em docs/excecoes-de-pre-condicao.md , exceções 1, 3, 4, 5, 6, 7) + 2 do invariante de catálogo. **Zero** "defeito de produto sem @bug ".

Isso muda o desenho da Etapa 1: o problema não é tag faltando em teste — é que a suíte tem uma **classe de resultado** (pré-condição ausente) que o runner só sabe expressar como *failed*, e o gate lê

failed como regressão. A correção é dar a essa classe um sinal estruturado e ensinar o gate a lê-lo. A recomendação "aplicar @bug aos 3-4 defeitos" do overview está **retirada**.

Segunda correção, menor: README.md registra **U-16** (logo 404 em 7 das 8 rotas de erros-de-console.spec.js , "regressão de 31/08"). Em 03/09 as 7 rotas passaram **sem** nenhuma exceção nova catalogada — U-16 não reproduziu. Entra na Etapa 2 como remedição, não como fato.

1. Decisões que dependem do dono do ambiente

Cinco pontos não são técnicos. Levantá-los **antes** de começar evita retrabalho:

#	Decisão	Bloqueia	Pergunta a fazer
D1	Catálogo onlyCanStart ganhou 6 processos para a conta T0TVS-FS : GestaoDependentes , SIGAJURI_AprovaFU , SIGAJURI_Contencioso , SIGAJURI_Contrato , rh_gbeneficios_planosaude , wf_substituicao cargos	Etapa 1.5	"Essa abertura de permissão foi intencional? É a mesma pergunta 1 de Perguntas em aberto (segregação de RH), agora com evidência do invariante"
D2	U-16 (logo 404) não reproduziu em 03/09	Etapa 2.2	"O branding foi corrigido, ou é intermitente?" — decide se sai da tabela ou vira exceção catalogada
D3	Retenção de relatórios: release do GitHub x artefato de CI x pasta compartilhada; e se o histórico do repositório interno deve ser reescrito	Etapa 3	"Onde o cliente vai consultar a evidência de cada execução?"
D4	Declarar no catálogo, como lacuna bloqueada, as regras de concorrência de fornecedores e de devolução na alçada	Etapa 8	"Vocês aceitam que a suíte diga 'não testado por D-01 + conta fora da SY1' em vez de omitir?"
D5	Job noturno de destrutivos continua com --workers=1 e passa a --retries=0	Etapa 5	comunicado, não pergunta — mas o dono pediu o ritmo de 60 s e precisa saber

2. Regras transversais — valem para toda etapa

Sem exceção, porque a suíte é software de produção (playwright-test-creator , princípio final):

1. **Nenhum test.skip , test.fixme , skip condicional ou test.fail() .** A decisão de não usar test.fail() está medida em docs/decisao-test-fail.md — ele não distingue defeito de pré-condição, e este ambiente já provou que a distinção importa.
2. **Nenhum teste é alterado para ficar verde.** Vermelho de produto continua vermelho.
3. **Toda mudança em teste é executada em primeiro plano** antes de ser dada como pronta (CLAUDE.md : "entregar sem execução comprovada não é"). Fatiado por arquivo se preciso.
4. **Prova de que reprova** (quality gate, item 5): o que for mecanismo de gate ou de alerta é testado nas duas direções — provoca-se a condição que deveria disparar e confirma-se que dispara; reverte-se e confirma-se que silencia.
5. **npm run typecheck e node --check limpos** ao fim de cada etapa.

6. **Nenhuma medição de cobertura ou determinismo fora de janela saudável:** `node scripts/sonda-grade.mjs` ≥ 840 em 5 amostras seguidas antes de qualquer número ir para documento.
7. **Um commit por etapa, direto na `main`** (convenção do projeto), mensagem explicando o *porquê* — o repositório já faz isso bem; manter.
8. **Números em documento citam o comando que os gera.** Foi a ausência dessa regra que produziu três totais diferentes para a mesma cobertura (Etapa 7).

3. Visão geral das fases

Fase	Etapas	Objetivo	Esforço
1 — Bloqueadores	1 · 2 · 3 · 4	gate utilizável, alarme confiável, repositório entregável	~1,5 dia
2 — Mecanismos	5 · 6 · 9	retries, seed por teste, cobertura auditável de verdade	~0,5 dia
3 — Catálogo e documentação	8 · 7	lacunas de negócio declaradas; docs alinhados ao código (por último, para refletir tudo)	~0,5 dia
4 — Valor ao cliente	10	recomendações de testabilidade + acessibilidade	~0,5 dia
5 — Medição final e entrega	11	execução completa em janela saudável, carimbo do gate, pacote	~0,5 dia (+ tempo de execução)

A ordem dentro das fases importa: **9 antes de 7** (a cobertura muda de número quando o script deixa de contar prosa), **2 antes de 7** (a contagem de `@bug` / `@achado` muda), **1 antes de 11**.

Fase 1 — Bloqueadores

Etapa 1 · Gate de CI que responde "há regressão nova?" (M-01)

Diagnóstico preciso

Escopo do gate: `PULAR_DESTRUTIVOS=1 --grep-invert "@bug|@achado"` → 136 testes, 9 vermelhos em 03/09, **nenhum deles regressão**. O gate está vermelho porque `PRÉ-CONDIÇÃO AUSENTE` chega ao runner como `throw new Error(...)` — 56 pontos em 24 specs e 4 utils — e o runner não tem como distinguir isso de assertion de produto. A convenção existe **na mensagem** (`docs/estabilidade-do-ambiente.md §"O que a suíte já faz"`) e no **registro formal** (`docs/excecoes-de-pre-condicao.md`), mas não em nada que uma máquina leia.

Três specs de RH já usam a anotação `type: 'pre-condicao-ausente'` (`dependentes.spec.js:45`, `substituicao-cargos.spec.js:42`, `admissao.spec.js:46`). A convenção já nasceu; falta generalizar.

Por que NÃO resolver com tag `@pre-condicao` nos 7 testes

Seria a correção de 1 hora, e é a que o overview sugeriu. Problemas:

- `CT-ACC-06-S2`, `D-02` e `CT-FAT-02-S2` são **guardas de regressão quando a massa existe**. Tirá-los do gate com uma tag apaga o sinal justamente no dia em que a massa aparecer e o produto regredir.
- Um mesmo teste é pré-condição hoje e defeito amanhã (`CT-FAT-02-S2` está na tabela de defeitos do README e falhou por massa em 03/09). Tag no título é estática; a causa não é.
- `modal-solicitacao-compra.spec.js:50` falhou por timeout de ambiente — não é estrutural, não merece tag nenhuma.

O critério correto é **classificar pelo resultado observado, nunca alterar o veredito do teste** — exatamente a linha de `docs/decisao-test-fail.md §4`, que já venceu a mesma discussão para o `@bug`.

1.1 · Criar `utils/pre-condicao.js`

```
// @ts-check
import { test } from '@playwright/test';

/** Tipo da anotação que o gate (`scripts/veredito-do-gate.mjs`) e o relatório leem. */
export const ANOTACAO_PRE_CONDICAO = 'pre-condicao-ausente';

/**
 * Declara que o cenário não pôde ser exercitado por falta de massa, serviço ou permissão —
 * e falha o teste, porque a skill proíbe skip e o relatório precisa mostrar o motivo no
 * caminho padrão de leitura.
 *
 * A anotação é o que permite ao gate separar "ambiente" de "regressão" sem ler mensagem.
 *
 * @param {string} motivo o que falta, quem destrava e por que NÃO é defeito do produto
 * @returns {never}
 */
export function faltaPreCondicao(motivo) {
  try {
    test.info().annotations.push({ type: ANOTACAO_PRE_CONDICAO, description: motivo });
  } catch {
    // fora de um teste (script de manutenção) não há relatório para anotar
  }
  throw new Error(`PRÉ-CONDIÇÃO AUSENTE: ${motivo}`);
}
```

`@returns {never}` faz o `checkJs` estreitar o tipo depois da chamada — remove os `if (!alvo) return;` // apenas para o `checkJs` que hoje existem em `criacao-solicitacao.spec.js:525` e similares.

1.2 · Migrar os 57 pontos de lançamento

Checklist mecânico (a verificação é o próprio `grep`):

- ☐ `utils/massa-contratos.js` (4 pontos, inclusive `MASSA INSUFICIENTE`, que é a mesma classe)
- ☐ `utils/espera-start.js`, `utils/captura-payload.js`, `utils/massa-medicao.js`
- ☐ 24 specs — `grep -rln "PRÉ-CONDIÇÃO AUSENTE" tests/`
- ☐ Padrão `expect(x, 'PRÉ-CONDIÇÃO AUSENTE: ...').toBeTruthy()` vira `if (!x) faltaPreCondicao('...')` — a mensagem não muda, o sinal nasce
- ☐ Os 3 specs de RH que já anotam à mão passam a usar o helper (evita duas grafias)

Aceite: `grep -rn "PRÉ-CONDIÇÃO AUSENTE" tests utils | grep -v "faltaPreCondicao\|pre-condicao.js"` devolve **0 linhas**.

1.3 · Criar `scripts/veredito-do-gate.mjs`

Lê o relatório JSON (não o JUnit — o JSON traz anotações e `status` por teste, incluindo `flaky`) e devolve o veredito que o gate precisa:

Classe	Critério	Bloqueia?
regressão	<code>status !== 'expected'</code> , sem <code>@bug / @achado</code> no título, sem anotação <code>pre-condicao-ausente</code>	sim — exit 1
pré-condição ausente	anotação presente	não — listado com o motivo
flaky	<code>status === 'flaky'</code> (passou no retry)	não — listado como "investigar" (arquitetura.md: retry é rede de segurança, nunca solução)
conhecido	<code>@bug / @achado</code> (não deveria estar no escopo; defesa)	não

Saída: tabela no console e em `GITHUB_STEP_SUMMARY`, no mesmo estilo de `scripts/alerta-bug-corrigido.mjs`. Sem dependência nova.

Pré-requisito: adicionar `['json', { outputFile: 'test-results/relatorio.json' }]` ao reporter de CI em `playwright.config.js`.

1.4 · Ajustar `.github/workflows/e2e.yml`, job `regressao`

```
- name: Suíte de regressão
  id: regressao
  continue-on-error: true          # o veredito é do passo seguinte, não do exit do runner
  run: PULAR_DESTRUTIVOS=1 npx playwright test --grep-invert "@bug|@achado"
  env: ...
- name: Veredito — há regressão nova?
  run: node scripts/veredito-do-gate.mjs test-results/relatorio.json
```

Mesmo padrão já usado no job `defeitos-conhecidos`. Atualizar o comentário de cabeçalho do job para descrever a nova semântica.

1.5 · Invariante de catálogo (`catalogo-invariante.spec.js:149` e `:224`) — depende de D1

O teste fez o que existe para fazer: 6 processos entraram no `onlyCanStart`. A correção **não é** atualizar a constante no reflexo (o próprio arquivo, linhas 28-33, proíbe). Sequência:

- ☐ Levar D1 ao dono do ambiente com a lista dos 6.
- ☐ Com a resposta: atualizar `INICIAVEIS_NO_CATALOGO` **com a data** e o motivo em comentário.
- ☐ Reescrever o teste da linha 224 (`SIGAJURI_Contencioso` fora do `onlyCanStart`) para a nova regra — a mensagem do próprio teste manda: *"reescrito para a nova regra — não silenciado"*.
- ☐ Rodar `tests/e2e/plataforma/inicio-processo-bloqueado.spec.js` e `tests/e2e/rh/bloqueio-processos-rh.spec.js`: a matriz de bloqueios duros e os `@achado` de RH precisam continuar coerentes com o catálogo novo.
- ☐ Se D1 disser "não foi intencional": o teste **fica vermelho** e vira defeito de segregação na tabela do README, com `@bug`.

1.6 · Fechar as linhas 5-8 de docs/excecoes-de-pre-condicao.md

Estão "□ em verificação" desde 01/09. As mensagens de 03/09 já trazem a investigação ao vivo ("*a base TEM cotações reais em aberto — 113002, 112860, 112839 — o que está vazio é o que a conta TOTVS-FS alcança*"). Evidência suficiente para "□ exceção confirmada", com data de revisão.

1.7 · Verificação da etapa

1. Rodar o escopo do gate localmente, fatiado (`tests/e2e/<área>` por vez, `--workers=4`) com `--reporter=json` → `node scripts/veredito-do-gate.mjs` → **exit 0**, com as pré-condições listadas (esperado: 5-7, conforme a massa do dia).
2. **Prova de que reprova:** inverter uma assertion em um teste sem tag (ex.: `home.spec.js`), rodar, veredito → **exit 1** nomeando o teste. Reverter.
3. **Prova de que a anotação chega ao relatório:** `npx playwright show-report` num teste que caiu em `faltaPreCondicao` mostra a anotação `pre-condicao-ausente` .
4. `npm run typecheck` .

Critério de aceite: o job `regressao` fica **verde** numa janela saudável, e a página de resumo do GitHub lista as pré-condições ausentes com motivo. `docs/estado-do-gate.md` "Recomendação 3 — portão de pré-condição" passa a "aplicado".

Etapa 2 · Alarme de @bug que só dispara quando deve (M-02)

2.1 · tests/e2e/plataforma/erros-de-console.spec.js:152

A tag está no template do `for` e alcança as 8 rotas; só o Portal do Comprador tem defeito catalogado (README, CT-PLT-06-S1).

```
/** Rotas cujo erro de console já está na tabela de defeitos do README. */
const ROTAS_COM_DEFEITO_CATALOGADO = new Set(['/portal/p/1/portal-do-comprador']);

for (const { nome, rota, titulo } of ROTAS_CHAVE) {
  const tag = ROTAS_COM_DEFEITO_CATALOGADO.has(rota) ? ' @bug' : '';
  test(`CT-PLT-06-S1${tag}: ${nome} (${rota}) deve carregar sem erro de console não catalogado`, ...
```

Atualizar o docstring do arquivo (§"Medição de 27/08/2026") para dizer que a tag é por rota.

2.2 · Remedir U-16 — depende de D2

- ☐ `npx playwright test tests/e2e/plataforma/erros-de-console.spec.js --repeat-each=3` em janela saudável.
- ☐ 0 de 3 com o logo 404 → README: U-16 passa a "não reproduzido desde 03/09/2026" (mantém histórico, muda estado). 1+ de 3 → intermitente: entra em `EXCECOES_CATALOGADAS` **só** com id, data e motivo, e o `casa` por recurso (`/portal/api/servlet/image/1/custom/logo_image.png`), nunca por texto.

2.3 · tests/e2e/portais/atribuicao-comprador.spec.js:36 → @achado

O teste afirma o comportamento **real** (a aba Atribuir não lista) e está verde. Pela tabela de CLAUDE.md e pelo critério "quem NÃO recebe @bug " do README ("teste verde que só exercita comportamento... que virariam vermelhos se ele mudasse"), é @achado .

- ☐ Trocar a tag no título.
- ☐ Corrigir o docstring do arquivo (linha 32: "CT-E2E-05-H fica vermelho por essa causa" — está verde; o vermelho é o segundo teste do arquivo, que cria massa).
- ☐ CLAUDE.md:227 "8 testes em 4 arquivos" → 9 em 5. README, contagem de @bug .

2.4 · Verificação

- PULAR_DESTRUTIVOS=1 npx playwright test --grep @bug --reporter=junit (fatiado) → node scripts/alerta-bug-corrigido.mjs test-results/junit.xml → **exit 0**, "Nenhum teste @bug passou".
- Prova de que dispara:** remover temporariamente o EXCECOES_CATALOGADAS do NPS 403 e rodar só home ... — simples demais; alternativa honesta: rodar --grep "CT-PLT-06-S1 @bug" e confirmar que **reprova** (Portal do Comprador), e depois o alerta com um JUnit em que esse caso esteja artificialmente verde (sed no XML) → **exit 1** nomeando-o.
- npx playwright test --grep @achado --list | tail -1 → 9.

Critério de aceite: o job defeitos-conhecidos termina sem ::warning:: numa execução normal.

Etapa 3 · Repositório entregável — evidência fora do versionamento (M-03)

Inventário

git ls-files | grep -i relatorio | grep -v '\.md\$' → **173 arquivos**, 277 MB: relatorio-2026-09-02/ (30), relatorio-2026-09-03/ (21), raiz (9, com .zip duplicado), relatorios/ e relatorios-2026-09-03/ (base.html 17 MB, merged.json 8 MB, 60+ JSON de fatia).

3.1 · Política de retenção — depende de D3

Recomendação, a confirmar:

Fica no repositório	Sai para <i>release asset</i> (gh release create execucao-2026-09-03 ...)
docs/execucoes/relatorio-falhas-<data>.md (a análise, ~240 KB)	.zip , .pdf , .html , pastas area-*.html , base.html , merged.json
docs/execucoes/<data>veredito.json — novo , saída de veredito-do-gate.mjs (~50 KB): status e classe por teste, o suficiente para recontar sem os 8 MB	JSON por fatia (ficam dentro do .zip)

O workflow já publica playwright-report/ como artefato por 30 dias; a release é o equivalente permanente e por data.

3.2 • Mover e ignorar

- ☐ `git mv relatorio-falhas-2026-09-0{2,3}.md docs/execucoes/`
- ☐ `git rm --cached` dos 173 – 2; publicar como assets **antes** de remover (a evidência não pode ficar sem casa entre um passo e outro).
- ☐ `.gitignore`: `relatorio-falhas*.{html,zip,pdf}`, `relatorio-20*/`, `relatorios*/`, `playwright-report*/`, `test-results*/` (a regra atual só cobre `test-results-0903/` literalmente).
- ☐ `scripts/relatorio-falhas.mjs`: conferir para onde grava e apontar para um diretório ignorado (`relatorios/`), com o `.md` final indo para `docs/execucoes/`.

3.3 • Histórico

Não reescrever a `main` agora. `git filter-repo` é irreversível, invalida todos os clones e worktrees, e o ganho é interno. Para o cliente, gerar o repositório de entrega **limpo** na Etapa 11 (branch órfã ou `git archive` do HEAD) — 0 risco, mesmo resultado. Se D3 pedir o repositório interno enxuto, agendar janela com todos os clones parados.

3.4 • Verificação

- `git ls-files | grep -i relatorio | grep -vc '\.md$' → 0`
- `git ls-files -z | xargs -0 du -ch | tail -1 → < 5 MB`
- `git check-ignore -v relatorio-falhas-x.zip relatorios-2026-09-04/merged.json → ambos casam`

Etapa 4 • `probe.mjs` e a regra do `.gitignore` (M-04)

- ☐ `git rm probe.mjs` — `scripts/sonda-grade.mjs` já faz o mesmo, sem caminho local e sem `catch{}` vazio.
 - ☐ `.gitignore`: `probe-*.mjs → probe*.mjs`.
 - ☐ Verificação: `git check-ignore -v probe.mjs` casa; `git ls-files | grep probe` vazio.
-

Fase 2 — Mecanismos

Etapa 5 · Destrutivos noturnos sem multiplicar massa (M-08) — comunicar D5

`playwright.config.js` define `retries: process.env.CI ? 2 : 0`; o job `destrutivos` herda. Cada vermelho vira até **3 registros** na base de homologação — 26 vermelhos em 03/09 → até 78 solicitações por noite.

☐ `.github/workflows/e2e.yml`, job `destrutivos`: `npx playwright test --grep @destrutivo --workers=1 --retries=0`

☐ Comentário do job: acrescentar o motivo (já está escrito no job `regressao`; referenciar).

☐ Verificação: `workflow_dispatch` manual → no relatório, nenhum teste com `retry #`. Confirmar no `test-results/criados.jsonl` que o número de SCs criadas = número de testes destrutivos que criam, não $\times 3$.

Regra da `cassi-fluig-master`: *"esperar por tempo é proibido, exceto para ritmo de escrita"* — o ritmo de 60 s continua na orquestração, não no código; `--workers=1` fica.

Etapa 6 · Seed do faker por teste, reprodução verdadeira (M-07)

Diagnóstico

`fixtures/fixtures.js:24` — `faker.seed(FAKER_SEED)` no carregamento do módulo. Cada worker é um processo; todos partem da mesma seed e consomem a sequência **na ordem de despacho**.

`reproduzirCom: FAKER_SEED=<v> npx playwright test` só é exato com `--workers=1`.

A `dados-ficticios.md` da skill descreve a seed **por execução** (é o modelo de referência); este projeto já foi além dela em `utils/massa-contratos.js` — identidade estável por teste, sem tag, com `repeatEachIndex` — e a seed deve seguir o mesmo princípio.

6.1 · Extrair a identidade do teste para reuso

☐ Criar `utils/identidade-do-teste.js` com `hash32` e `idEstavelDoTeste(testInfo)` (título sem `@tags` + `repeatEachIndex`, **sem** `retry` — a tentativa reproduz), movidos de `utils/massa-contratos.js`, que passa a importar de lá.

6.2 · Semear na fixture `evidence`, antes do `use`

```
// fixtures/fixtures.js – dentro de `evidence`, antes de `await use(undefined)`
const idDoTeste = idEstavelDoTeste(testInfo);
const seedDoTeste = (FAKER_SEED ^ hash32(idDoTeste)) >>> 0;
faker.seed(seedDoTeste);
testInfo.annotations.push({ type: 'faker-seed', description: String(FAKER_SEED) });
testInfo.annotations.push({ type: 'faker-seed-do-teste', description: String(seedDoTeste) });
```

Seguro porque cada worker executa um teste por vez (mesmo argumento de `reservasEmPosse`). O `faker.seed(FAKER_SEED)` global fica, para uso fora de teste. Atualizar os docstrings de `factories/*.js` ("fixada por execução" → "por execução e por teste").

6.3 · Verificação

1. `FAKER_SEED=123 npx playwright test tests/e2e/rh/dependentes.spec.js tests/e2e/rh/admissao.spec.js --workers=4 --repeat-each=3` **duas vezes**. Comparar a anotação `massa-pronta-para-uso-futuro` (que já embute a saída da factory) entre as execuções, por teste e `repeatEachIndex` → **idênticas**.
2. Repetir invertendo a ordem dos arquivos → idênticas (prova que a ordem de despacho não entra mais).
3. **Prova de que era um problema:** rodar o mesmo comando no commit anterior à mudança → valores diferem entre as duas execuções.

Etapa 9 · Cobertura auditável sem "menção em prosa" (M-09)

Diagnóstico — dois problemas, não um

(a) O script tem um defeito de ordem. `scripts/gerar-cobertura.mjs:119-124`:

```
if (!declarados.has(id)) soEmProsa.add(id);
else soEmProsa.delete(id);
```

O veredito é do **último arquivo processado** que menciona o ID. `CT-PLT-10-H` está no título em `catalogo-invariante.spec.js:149` e em prosa em `processo-inativo-e-residuo.spec.js:46` — processado depois, reinsere no conjunto. Dos 7 avisados, **4 são só isso** (`CT-CMP-02-S4`, `CT-DEP-02-S1`, `CT-PLT-09-S1`, `CT-PLT-10-H`).

(b) **Prosa conta como cobertura.** `onde` é montado de `fonte.match(RE)` — qualquer menção. O docstring (linhas 92-105) assume o custo conscientemente, mas ele produz inconsistência: `CT-DEP-01-H` está "coberto" porque uma mensagem de assertion o cita, enquanto `CT-DEP-01-S1/S2/S3` — mesmo bloqueio, mesma frase — são lacunas com motivo. Idem `CT-SUB-01-H` × `CT-SUB-01-S1/S2`.

9.1 · Corrigir o script

- ☐ Manter um conjunto `declaradoEmAlgumArquivo` acumulado; `soEmProsa = mencionados - declarados`.

- ☐ **Cobertura = título u cabeçalho do arquivo.** Prosa deixa de contar. O aviso "contados por menção em prosa" desaparece por construção, não por limpeza.
- ☐ Rodar. Cada ID que **cair** para lacuna recebe uma de duas ações — nunca uma terceira:

Situação	Ação
existe teste que o exercita (ou documenta o bloqueio)	levar o ID completo ao título (CT-DEP-01-S1 — a regex não casa 01-S1)
não existe	entrada em MOTIVOS

9.2 · Os três casos que sobram

- CT-CMP-02-S2 — validacoes-solicitacao-compras.spec.js:49 é docstring sobre um teste sem o ID no título → **título**.
- CT-DEP-01-H, CT-SUB-01-H — coerência com os irmãos: **MOTIVOS**, mesmo texto dos S1/S2/S3 ("o formulário não monta campo sem matrícula ativa").

9.3 · Verificação

- npm run cobertura → passa; o documento traz *"Nenhum caso é contado por menção em prosa"*.
 - Anotar o número novo — vai para a Etapa 7.
-

Fase 3 — Catálogo e documentação

Etapa 8 · Regras de negócio ausentes viram lacunas declaradas (M-06) — depende de D4

A `cassi-fluig-master` (`regras-de-negocio-compras.md`) chama a regra de concorrência de *"uma das regras mais testáveis do ciclo"* e os cinco caminhos de devolução na alçada de *"cobertura de regressivo óbvia e hoje inexistente"*. O catálogo não os tem; a matriz de cobertura, portanto, não sabe que faltam.

8.1 · Novos casos em `docs/catalogo-casos.md` (formato das entradas existentes, ver `CT-CMP-05-H` na linha 329)

ID	Caso	Fonte (skill)	Motivo do bloqueio hoje
<code>CT-C0T-03-H</code>	Com dispensa: exatamente 1 fornecedor é aceito	§4, "Regra dura de concorrência"	D-01 (nenhuma SC da automação chega à Cotação) e <code>T0TVS-FS</code> fora da SY1 (<code>catalogo-casos.md:973</code>)
<code>CT-C0T-03-S1</code>	Sem dispensa, 2 fornecedores → recusado	idem	idem
<code>CT-C0T-03-S2</code>	Com dispensa, 2 fornecedores → recusado	idem	idem
<code>CT-CMP-05-S2</code>	Alçada: alteração client-side do payload de aprovação é rejeitada pelo servidor	§8, "trava rígida... Auditoria Interna"	conta fora da AL/DHL. Nota: a skill (§10) já aponta <code>CT-SEG-07-S1</code> como a divergência medida dessa regra — citar a relação no caso
<code>CT-CMP-06-S1...</code> <code>S5</code>	Os 5 retornos da alçada (regerar documento, novo fornecedor, cotação, negociação, cancelar)	§8, "Devolução na Alçada"	idem — segundo lote, se D4 aceitar

Arredondamento fiscal / *saving*: **não** vira caso. Fato 1 da skill — *"o Fluig não é dono de regra financeira"* —; a suíte afirma coerência **interna** do payload (`CT-ACC-08-S1/S2`), que é o que o Fluig controla. Registrar essa decisão numa nota no cabeçalho da seção `CT-ACC` .

8.2 · `scripts/gerar-cobertura.mjs` → **MOTIVOS** para cada ID novo

O script **reprova** se um caso ficar sem teste e sem motivo — é o mecanismo que garante que a lacuna não envelhece em silêncio. Usar isso a favor: acrescentar os casos, rodar, deixar o script listar o que falta, preencher.

8.3 · Verificação

- `npm run cobertura` passa; as linhas novas aparecem □ com motivo.
- `README.md` §"Backlog": nova linha na tabela de motivos — "Consequência de defeito aberto + cadastro no ERP (SY1/AL)".

Etapa 7 · Documentação alinhada ao código (M-05) — por último na fase

Cada linha abaixo é uma edição pontual; a fonte do número está ao lado. **Regra:** número em prosa vem de comando, não de memória.

Arquivo:linha	Hoje	Passa a	Fonte		
docs/estado-do-gate.md (topo)	65 specs · 177 testes · 36 POs · "25/08"	81 · 233 · 41 · seção nova "Execução completa 03/09/2026" com a classificação de docs/execucoes/relatorio-falhas-2026-09-03.md	`find tests -name '*.spec.js' \	wc -l ; ls pages \	wc -l ; relatorios-2026-09-03/derivados.mjs`
docs/estado-do-gate.md §Recomendações	3 e 4 pendentes	3 aplicado (Etapa 1)	—		
CLAUDE.md:292	163 casos / 132 cobertos	número da Etapa 9	npm run cobertura		
CLAUDE.md:227	8 @achado em 4 arquivos	9 em 5	--grep @achado --list		
README.md:138-141	163 / 133 / 30 / 34 destrutivos	Etapa 9 / 45 destrutivos	--grep @destrutivo --list		
README.md:143	"última execução 26/08 — 178 testes"	03/09 — 233 / 162 / 71	relatório		
README.md:408	"os 30 casos sem teste"	número da Etapa 9	—		
README.md §"A tag @bug "	"66 testes"	número após a Etapa 2	--grep @bug --list		
README.md §"Perguntas em aberto", item 2	descreve o default de factory já corrigido	reescrever: lista confirmada pelo dono em 31/08; a factory não tem default; modal-solicitacao-compra.spec.js é o guardião	factories/solicitacao-compra.js		
README.md tabela de defeitos, U-16	"regressão de 31/08"	estado após D2	Etapa 2.2		
docs/excecoes-de-pre-condicao.md linhas 5-8	□ em verificação	□ confirmada	Etapa 1.6		
README.md §CI	uma frase	descrever os três jobs e o veredito (Etapa 1)	—		

Verificação: um `grep -n` por cada número antigo (163 , 132 , 133 , 177 , "30 casos" , "66 testes")
devolve zero ocorrências em `CLAUDE.md` , `README.md` e `docs/` .

Fase 4 — Valor ao cliente

Etapa 10 · docs/recomendacoes-de-testabilidade.md (M-10)

A `playwright-test-creator` manda "*recomendar data-testid ao time de desenvolvimento em vez de criar seletor frágil*". O repositório contorna bem (491 `getByRole`, CSS só onde não há papel), mas nunca devolveu a lista. `docs/mapa-do-ambiente.md:173-181` já tem três itens de acessibilidade — este documento os absorve e completa.

10.1 · Três grupos, porque o destinatário é diferente

Grupo	Exemplos (de pages/)	Quem resolve	Recomendação
Plataforma TOTVS — não alterável pelo cliente	<code>task-card-component</code> , <code>.panel-task-chart-*</code> (<code>CentralTarefasPage.js:45-54</code>), <code>#ecm-documentPublisher-documentDesc</code> , <code>#inputFile</code> (<code>DocumentosGedPage.js:89-90</code>), <code>#dd-options / #optionList</code> (<code>AcoesDaTarefaPage.js:48-49</code>), <code>.select2-*</code> , <code>.tt-input</code> (<code>CicloCompradorPage.js:437-459</code>)	TOTVS	aceitar como CSS estável (prioridade 6 da skill); registrar a versão do Fluig em que foram medidos
Formulários do cliente — ids já estáveis	<code>input#WKNumProces</code> , <code>#emailSolicitante</code> , <code>textarea#obsRejeicao</code> (<code>FormularioRejeicoesPagamentosPage.js:68-82</code>), <code>#substitutoDtInicial</code> (<code>DelegacaoTarefasPage.js:85</code>), <code>#tbprod_*__1</code> (<code>CicloCompradorPage.js:214-221</code>), <code>#clinica</code> (<code>QuestionarioClinicassiPage.js:43</code>)	time Fluig da Cassi	<code><label for></code> em todo campo → a suíte migra para <code>getByLabel</code> (prioridade 2). Ganho: acessibilidade real, não só teste. Manter os ids
Widgets do cliente — Angular/PO-UI	ícones da coluna Ação sem nome (<code>title</code> como gancho), <code>.tooltip-inner</code> interceptando clique (<code>PreenchimentoSolicitacaoCompraPage.js:38,108</code> , <code>MedicaoContratoPage.js:98-102</code>), <code>po-icon</code> de detalhe (<code>CicloCompradorPage.js:179</code>), abas da Home com <code><div></code> dentro de <code><a></code>	time Fluig da Cassi	<code>aria-label</code> nos ícones; <code>pointer-events: none</code> no tooltip; <code>data-testid</code> onde não houver papel semântico; corrigir o HTML das abas

10.2 · Um achado cruzado com a `fluig-master` que vale sozinho o documento

Fato 7 da skill: "*a 2.0 (Voyager) quebra front-end customizado — paths de CSS retornam 404*". O erro catalogado do Portal do Comprador (CT-PLT-06-S1) é exatamente `404 /style-guide/css/fluig-style-guide.min.css`. Não é bug isolado: é o sintoma documentado de widget **pré-Voyager rodando em plataforma 2.0**. Recomendação ao cliente: revisar `wg_portalCompradores` contra `references/voyager-2-migracao.md` — e provavelmente os demais widgets customizados, antes que o próximo update quebre mais coisa.

10.3 · Formato de cada linha

Onde (rota/tela) · Elemento · Como a suíte o alcança hoje · Por que é frágil · Atributo recomendado · Ganho (ally / teste) · Prioridade . Curto — cabe em 2 páginas.

Fase 5 — Medição final e entrega

Etapa 11 · Carimbo do gate e pacote

1. **Janela saudável:** `node scripts/sonda-grade.mjs` × 5 → ≥ 840 em todas. SIGAJURI conferido separadamente (`docs/estabilidade-do-ambiente.md` §protocolo, item 4).
2. **Execução completa, fatiada, em primeiro plano** — não-destrutivos em paralelo por área; `@destrutivo` um por invocação com 60 s (regra do dono). `--retries=0` nos destrutivos.
3. `node scripts/veredito-do-gate.mjs` sobre o merge → **0 regressões**; registrar o número de pré-condições e de flaky.
4. `node scripts/alerta-bug-corrigido.mjs` → exit 0 (ou a lista real de defeitos corrigidos).
5. `npm run cobertura`, `npm run typecheck`, varredura do quality gate §2 da skill.
6. `docs/estado-do-gate.md`: seção "Carimbo — <data>" com os números, o comando de cada um, e a frase que hoje falta: *"o gate de regressão está verde; os N vermelhos restantes são defeito catalogado (@bug), achado versionado (@achado) ou pré-condição registrada em docs/excecoes-de-pre-condicao.md"*.
7. Relatório da execução: `.md` em `docs/execucoes/`, evidência pesada como release asset (D3).
8. Repositório de entrega limpo (branch órfã / `git archive`), sem os 277 MB de histórico.

Definição de pronto: os 12 itens do quality gate da skill conferidos, os 3 que estavam com ressalva na revisão (`determinismo`, `CI`, `seed`) agora plenos, e nenhuma decisão D1–D5 pendente sem resposta registrada.

Apêndice A — Checklist consolidado

- ☐ **D1–D5** levantados com o dono do ambiente
- ☐ **E1** `utils/pre-condicao.js` · 57 migrações · `veredito-do-gate.mjs` · workflow · invariante (D1) · exceções 5–8 · provas nas duas direções
- ☐ **E2** tag por rota · U-16 remedido (D2) · `atribuicao-comprador` → `@achado` · alerta exit 0
- ☐ **E3** política (D3) · assets publicados · `git rm --cached` · `.gitignore` · script de relatório apontando para pasta ignorada · tracked < 5 MB
- ☐ **E4** `probe.mjs` removido · `probe*.mjs`
- ☐ **E5** `--retries=0` no job noturno (D5)
- ☐ **E6** `utils/identidade-do-teste.js` · seed por teste · prova de reprodução sob `--workers=4`
- ☐ **E9** script sem defeito de ordem · prosa não conta · IDs em título ou `MOTIVOS`
- ☐ **E8** casos novos no catálogo (D4) · `MOTIVOS` · README backlog
- ☐ **E7** todas as linhas da tabela da Etapa 7 · grep dos números antigos = 0
- ☐ **E10** `docs/recomendacoes-de-testabilidade.md` com os três grupos e o achado Voyager
- ☐ **E11** janela saudável · execução completa · veredito 0 regressões · carimbo · pacote

Apêndice B — O que este plano NÃO faz, de propósito

- Não migra `@bug` para `test.fail()` — decisão medida em `docs/decisao-test-fail.md`.
- Não "conserta" nenhum dos 71 vermelhos — 54 são defeito catalogado, 8 achado, os demais pré-condição; todos continuam vermelhos até o produto ou o ambiente mudar.
- Não reescreve a história do repositório interno sem D3.
- Não implementa os casos de concorrência/alçada — declara-os, porque a conta da automação não alcança a etapa (D-01 + SY1/AL). Implementar exige provisionamento que compete ao cliente.